

```
In [ ]: import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
```

```
In [ ]: # Download historical stock data
ticker = 'ICICIBANK.NS'
data = yf.download(ticker, start='2020-01-01', end='2024-07-24')
```

[*****100%*****] 1 of 1 completed

```
In [ ]: data
```

```
Out[ ]:
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2020-01-01	539.900024	542.500000	535.700012	536.750000	523.320801	7642793
2020-01-02	536.000000	541.900024	535.849976	540.599976	527.074341	10264223
2020-01-03	538.000000	540.799988	536.250000	538.849976	525.368164	9594690
2020-01-06	537.250000	537.250000	523.799988	525.700012	512.547241	12534334
2020-01-07	531.299988	535.599976	521.650024	522.900024	509.817261	16101835
...	...	...	...	...	...	...
2024-07-16	1230.000000	1245.000000	1229.000000	1239.900024	1239.900024	12320605
2024-07-18	1234.900024	1256.449951	1231.699951	1250.300049	1250.300049	10999632
2024-07-19	1244.000000	1250.500000	1240.250000	1248.650024	1248.650024	7958817
2024-07-22	1240.000000	1245.300049	1232.900024	1241.150024	1241.150024	9958965
2024-07-23	1241.150024	1248.650024	1212.500000	1226.750000	1226.750000	19369832

1128 rows × 6 columns

```
In [ ]: # Calculate RSI
def calculate_rsi(data, window = 14):
    delta = data['Close'].diff()
    gain = (delta.where(delta>0,0)).rolling(window = window).mean()
    loss = (-delta.where(delta<0,0)).rolling(window = window).mean()
    rs = gain/loss
    rsi = 100 - (100/(1+rs))
    return rsi

In [ ]: data['RSI'] = calculate_rsi(data)

In [ ]: def fast_sma(data, window = 21):
    return data['Close'].rolling(window).mean()

def slow_sma(data, window = 100):
    return data['Close'].rolling(window).mean()

In [ ]: data['fast_sma_price'] = fast_sma(data)
data['slow_sma_price'] = slow_sma(data)

In [ ]: # Implement the strategy
def implement_rsi_strategy(data):
    buy_signal = []
    sell_signal = []
    position = False
    for i in range(len(data)):
        if data["RSI"].iloc[i]>30 and data['fast_sma_price'][i] > data['slow_sma_price'][i-1] and position==False:
            buy_signal.append(data['Close'].iloc[i])
            sell_signal.append(np.nan)
            position = True
        elif data['RSI'].iloc[i] > 70 and data['fast_sma_price'][i] < data['slow_sma_price'][i-1] and position==True:
            buy_signal.append(np.nan)
            sell_signal.append(data['Close'].iloc[i])
            position = False
        else:
            buy_signal.append(np.nan)
            sell_signal.append(np.nan)

    return buy_signal, sell_signal
```

```
In [ ]: data["Buy_price_signal"], data["Sell_price_signal"] = implement_rsi_strategy(data)
```

C:\Users\Amit.Ranjan\AppData\Local\Temp\ipykernel_35032\1876612905.py:7: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
 if data["RSI"].iloc[i]>30 and data['fast_sma_price'][i] > data['slow_sma_price'][i-1] and position==False:
 C:\Users\Amit.Ranjan\AppData\Local\Temp\ipykernel_35032\1876612905.py:11: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
 elif data['RSI'].iloc[i] > 70 and data['fast_sma_price'][i] < data['slow_sma_price'][i-1] and position==True:

```
In [ ]: import numpy as np

# Filtering not equal to Nan values in buy price signal column

data1 = data[~data['Buy_price_signal'].isna()]
data1
```

```
Out [ ]:
```

	Open	High	Low	Close	Adj Close	Volume	RSI	fast_sma_price	slow_sma_price	E
Date										
2020-07-24	388.299988	388.700012	372.149994	381.799988	372.247528	52980980	59.326781	362.354761	358.442500	
2022-07-18	759.250000	773.150024	757.849976	772.000000	754.888794	8045363	75.432844	725.911909	722.605497	
2023-04-24	905.000000	906.500000	895.000000	904.750000	890.005432	30881665	71.965699	876.661912	876.209004	
2023-12-13	1014.799988	1020.700012	1009.200012	1013.900024	1013.900024	15887040	81.748544	960.347618	959.207500	

```
In [ ]: # Plot the results
plt.figure(figsize=(12,6))
plt.plot(data['Close'], label='Close Price', alpha=0.5)
plt.scatter(data.index, data['Buy_price_signal'], label='Buy Signal', marker='^', color='green', alpha=1)
plt.scatter(data.index, data['Sell_price_signal'], label='Sell Signal', marker='v', color='red', alpha=1)
plt.title(f'{ticker} Price History with Buy and Sell Signals')
```

```
plt.xlabel('Date')  
plt.ylabel('Close Price USD ($)')  
plt.legend(loc='best')  
plt.show()
```

